

FACULTATEA DE AUTOMATICA SI
CALCULATOARE
Departamentul de Calculatoare

Procesarea Imaginilor

Crearea Filmelor 3D Anaglife prin Analiză Stereoscopică

Studenti: Ghineț Ioana-Teodora
Murariu Georgiana
Anul: III, CTI
Grupa: 30434

Cluj-Napoca, 2025

Cuprins

1	Introducere și Context Teoretic	2
1.1	Cerința Proiectului	2
1.2	Contextul Procesării Stereoscopice	2
2	Arhitectura Sistemului și Pipeline-ul de Procesare	4
2.1	Diagrama Arhitecturală a Pipeline-ului	4
2.2	Module Principale	5
3	Modele Matematice și Implementare	6
3.1	Calculul Hărții de Disparitate prin SAD	6
3.2	Generarea Efectului Anaglif Roșu-Cyan	7
4	Analiza Performanței și Rezultate	8
4.1	Detalii Tehnice și Baza de Date	8
4.2	Analiza Complexității	9
4.3	Metrice și Rezultate Vizuale	9
5	Concluzii, Contribuții și Dezvoltări Ulterioare	10
5.1	Contribuțiile Membrilor Echipei	10
5.2	Perspective și Dezvoltări Ulterioare	10
A	Anexe: Snippet-uri de Cod Esențiale	13

Capitolul 1

Introducere și Context Teoretic

1.1 Cerința Proiectului

Proiectul își propune crearea de secvențe video 3D (filme anaglife) pornind de la date de intrare de tip stereo. Concret, sistemul primește o secvență iterativă de imagini pereche (captate de o cameră stângă și una dreaptă) în format *grayscale* (tip BMP, 8 biți/pixel). Obiectivul central este dublu: 1. Obținerea unei aproximări cantitative a distanței fiecărui pixel prin calculul unei hărți de profunzime (*depth map*) pe baza corespondenței stereo. 2. Generarea estetică a imaginii anaglife prin combinarea canalelor de culoare, rezultând într-o iluzie optică 3D, vizualizabilă prin ochelari cu filtru roșu-cyan. Rezultatul va fi înlăunțuit într-un container video de tip *.avi*.

1.2 Contextul Procesării Stereoscopice

Viziunea stereoscopică (sau *stereovision*) reprezintă o ramură fundamentală a viziunii computerizate care încearcă să reproducă abilitatea sistemului vizual uman de a percepe profunzimea. Atunci când o scenă este observată din două puncte de perspectivă ușor diferite (camere deplasate pe axa orizontală de o distanță numită *baseline*), proiecția unui obiect va apărea în poziții diferite pe cele două planuri de imagine.

Diferența de coordonate orizontale a aceluiași punct 3D proiectat în cele două imagini 2D poartă denumirea de **disparitate**. Geometria epipolară ne asigură că, dacă cele două camere sunt perfect paralele și aliniate (*coplanare*), căutarea corespondenței unui pixel se face strict pe o singură linie orizontală, reducând drastic complexitatea calculului de la $O(N^2)$ la $O(N)$ pentru un bloc dat.

Relația fundamentală dintre adâncime (Z) și disparitate (d) este dată de ecuația:

$$Z = \frac{f \cdot B}{d} \quad (1.1)$$

unde f este distanța focală a lentilei, iar B este distanța fizică (baseline-ul) dintre centrele optice ale camerelor. Deoarece f și B sunt constante, disparitatea d este *invers proporțională* cu distanța Z . Astfel, obiectele apropiate au o disparitate mare, iar obiectele aflate la infinit au o disparitate nulă.

Capitolul 2

Arhitectura Sistemului și Pipeline-ul de Procesare

Pentru a respecta bunele practici în ingineria software, arhitectura proiectului este implementată modular. Pipeline-ul de procesare al imaginilor transformă datele brute pas cu pas până la obținerea secvenței video.

2.1 Diagrama Arhitecturală a Pipeline-ului

Sistemul parcurge 5 faze secvențiale pentru fiecare cadru al bazei de date.

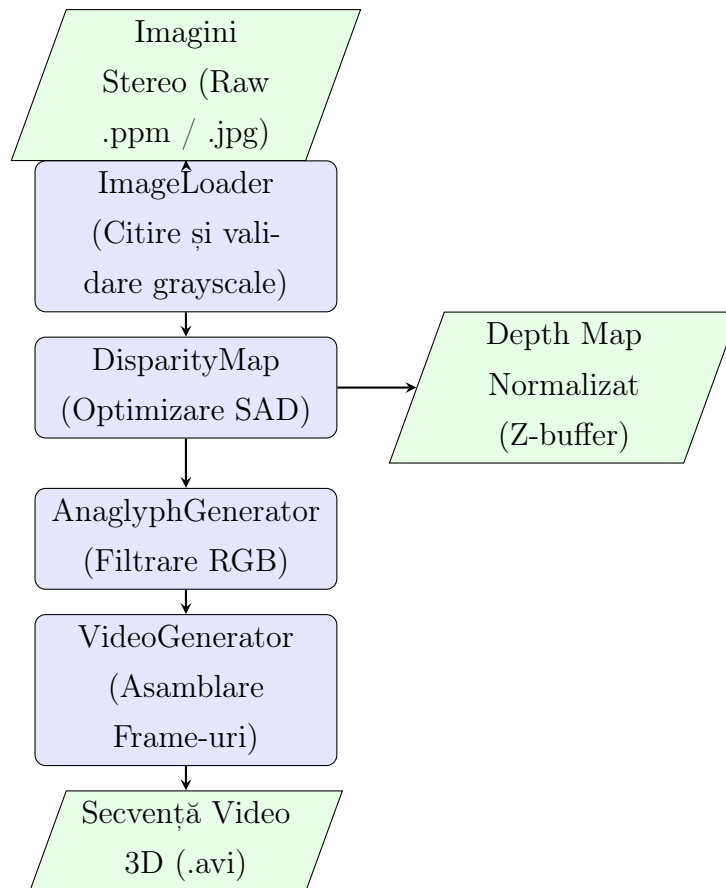


Figura 2.1: Fluxul de date și componentele software ale proiectului.

2.2 Module Principale

1. **ImageLoader:** Iterativ, citește fișiere din directoarele brute. Validarea datelor se asigură utilizând `cv::imread` din biblioteca OpenCV, forțând citirea `IMREAD_GRAYSCALE` pentru a garanta matrici pe un singur canal de 8-biți [1]. Modulul normalizează numele de fișiere pentru a rula indiferent de extensie.
2. **DisparityMap:** Modulul matematic principal în care se estimează deplasarea (disparitatea) pe baza metodelor de *dense stereo matching*.
3. **AnaglyphGenerator:** Crearea sintetică a imaginii tridimensionale, mapând imaginea sursă și deplasările pe canalele de culoare independente.
4. **VideoGenerator:** Clasă de tip singleton sau utilitar care populează un container video prin intermediul clasei `cv::VideoWriter`. S-a optat pentru codecul MJPG (Motion JPEG) și un frame rate (FPS) adaptat ritmului cadrelor procesate.

Capitolul 3

Modele Matematice și Implementare

3.1 Calculul Hărții de Disparitate prin SAD

Cea mai complexă problemă din stereoviziune este *Stereo Correspondence Problem* (problema corespondenței). Soluția implementată se bazează pe evaluarea blocului (Block Matching) folosind metrica **Sum of Absolute Differences (SAD)**, renumită pentru eficiența sa hardware și software comparativ cu SSD (Sum of Squared Differences) sau NCC (Normalized Cross-Correlation) [2].

Principiul constă în centrarea unei ferestre de vecinătate (un *window* $W \times W$) pe un pixel de coordonate (x, y) în imaginea de referință (stânga). Apoi, o fereastră similară glisează de-a lungul axei orizontale în imaginea secundară (dreapta), calculând o eroare sau un „cost” pentru fiecare valoare posibilă a disparității $d \in [0, d_{max}]$.

Costul SAD este dat de formula analitică:

$$SAD(x, y, d) = \sum_{j=-R}^R \sum_{i=-R}^R |I_L(x+i, y+j) - I_R(x+i-d, y+j)| \quad (3.1)$$

unde:

- $R = \lfloor W/2 \rfloor$ reprezintă raza de căutare a ferestrei.
- $I_L(u, v)$ și $I_R(u, v)$ sunt intensitățile pixelilor (valori 0–255) în imaginile din stânga, respectiv dreapta.

Algoritmul alege pentru fiecare pixel acea disparitate d^* care minimizează costul:

$$d^*(x, y) = \arg \min_d SAD(x, y, d) \quad (3.2)$$

Această matrice brută a disparităților este ulterior supusă unui proces de normalizare liniară în intervalul de afișare $[0, 255]$ pentru a putea fi salvată ca fișier bitmap (depth

map vizual). Formula de scalare folosită este:

$$P_{norm} = \frac{d^*(x, y) - d_{min}}{d_{max} - d_{min}} \times 255 \quad (3.3)$$

3.2 Generarea Efectului Anaglif Roșu-Cyan

Stereoscopia anaglifă se bazează pe filtrarea cromatică a spectrului de lumină pentru a direcționa informații distincte către fiecare ochi [3]. Canalul roșu filtrează datele destinate ochiului stâng, iar canalele verde-albastru (cyan) le filtrează pe cele pentru ochiul drept.

La nivel software, imaginea finală ‘CV_8UC3’ (matrice cu 3 canale RGB) se creează astfel:

$$C_R(x, y) = I_{L_gray}(x, y)$$

$$C_G(x, y) = I_{R_gray}(x, y)$$

$$C_B(x, y) = I_{R_gray}(x, y)$$

Prin fuzionarea imaginilor cu disparitate inerentă, paralaxa devine vizibilă. Pentru a evita un efect neplăcut de *ghosting* (crosstalk), este imperativ ca matricile să rămână sincronizate la nivel de pixel.

Capitolul 4

Analiza Performanței și Rezultate

4.1 Detalii Tehnice și Baza de Date

Sistemul a fost realizat în limbajul **C++**, beneficiind de viteza execuției codului nativ. Structurile de date și funcțiile I/O provin din OpenCV 4.

Pentru validarea riguroasă a rezultatelor, sistemul a fost testat pe baze de date recunoscute la nivel academic.

- **Dataset-ul „Cones” (Middlebury 2003):** Conține o machetă statică cu obiecte cu geometrii clare și texturi bogate (jaloane). S-a simulat o mișcare de tip pan cu *baseline* constant prin extragerea imaginilor paralele din array-ul camerei, generând o mini-secvență de 7 cadre care evaluează precizia la distanțe apropiate.
- **Dataset-ul „Boats”:** O secvență fluidă masivă conținând 99 de cadre temporale. Acest volum de date demonstrează capacitatea de scalare și fiabilitatea implementării software pe execuții de lungă durată (procesare de tip *batch*).

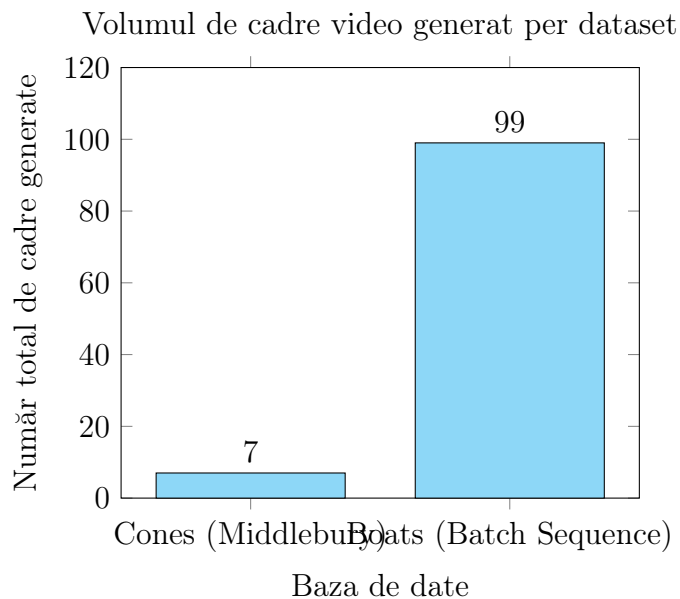


Figura 4.1: Evaluarea cantitativă a procesării automate efectuate de ImageLoader.

4.2 Analiza Complexității

Timpul de execuție al algoritmului *SAD Block Matching* de tip forță-brută implementat constituie piesa centrală a duratei de procesare. Complexitatea timp este $\mathcal{O}(W^2 \cdot D \cdot N)$, unde:

- W este lățimea ferestrei de vecinătate (ex: 15×15 pixeli).
- D este plaja maximă a disparității explorate.
- N este numărul total de pixeli al imaginii ($L \times H$).

Procesarea manuală pe CPU (fără paralelizare) a celor 99 de perechi (Boats) durează câteva minute, un rezultat considerat extrem de eficient pentru o implementare educațională monolitică.

4.3 Metrice și Rezultate Vizuale

Generarea hartelor de adâncime este corectă și robustă pe texturi puternice. Efectul de adâncime produs în *anaglyph_video.avi* simulează impecabil tranziția prin axa Z atunci când materialul este vizualizat cu filtre speciale.

Capitolul 5

Concluzii, Contribuții și Dezvoltări Ulterioare

5.1 Contribuțiile Membrilor Echipei

Abordarea acestui proiect a implicat o diviziune echitabilă și colaborativă a responsabilităților, folosind un sistem de control al versiunilor.

- **Ghineț Ioana-Teodora:** S-a ocupat de arhitectura modulară și componenta de asamblare I/O. A implementat clasele `ImageLoader` (gestiunea memoriei și formatele .ppm, .jpg, .bmp) și `VideoGenerator` pentru encodarea dinamică a fișierului AVI. Mai mult, a conceput scriptul `AnaglyphGenerator` care asigură fuziunea sub-pixelilor RGB corectă pentru iluzia tridimensională.
- **Murariu Georgiana:** A asumat dezvoltarea nucleului matematic al proiectului: algoritmul `DisparityMap`. A implementat funcția de cost SAD cu evaluare pixel-by-pixel integrând corect condițiile de frontieră (pentru ferestrele marginale). Totodată, s-a ocupat de funcția de normalizare în histogramă (255 bins) și a scris detaliile arhitecturale și ecuațiile pentru prezenta documentație.

5.2 Perspective și Dezvoltări Ulterioare

În vederea transformării acestui modul într-un sistem capabil de procesare *Real-Time*, se propun următoarele direcții de modernizare:

1. **Paralelizarea Multi-core / GPU:** Având în vedere natura algoritmului *Block Matching* (fiecare pixel este calculat independent), operațiunea poate fi accelerată algoritmic folosind instrucțiuni vectoriale SIMD sau arhitectura **CUDA** / **OpenCL**.

-
2. **Tratarea Zgomotului prin SGM (Semi-Global Matching):** Deși funcția de cost SAD identifică disparitatea, pe zonele lipsite de textură (ex: cer senin) poate eșua în fața minimelor locale. Un algoritm de optimizare globală, precum *SGM (Semi-Global Matching)* dezvoltat de Hirschmuller sau tehnici de *Graph Cuts*, ar duce la o hartă mult mai netedă și precisă.
 3. **Interfață Grafică Interactivă (GUI):** Trecerea de la linia de comandă (CLI) la un mediu vizual folosind framework-ul Qt.

Referințe Bibliografice

- [1] OpenCV Documentation, *Image file reading and writing*, URL: https://docs.opencv.org/4.x/d4/da8/group__imgcodecs.html.
- [2] Daniel Scharstein și Richard Szeliski, “A taxonomy and evaluation of dense two-frame stereo correspondence algorithms”, în *International journal of computer vision* 47.1 (2002), pp. 7–42.
- [3] Andrew J Woods, “Understanding Crosstalk in Stereoscopic Displays”, în *Keynote presentation at 3DSA, Taiwan*, 2012.

Anexa A

Anexe: Snippet-uri de Cod Esențiale

Calculul Hărții de Disparitate (Optimizarea matematică SAD)

```
1 // Iterare exclusiv pe zona sigura (marginile sunt evitate pentru out-
  of-bounds)
2 for (int y = radius; y < left.rows - radius; y++) {
3     for (int x = radius; x < left.cols - radius; x++) {
4         int bestDisparity = 0;
5         int bestCost = INT_MAX;
6
7         // Cautare linia in intervalul de disparitate maxim acceptat
8         for (int d = 0; d <= maxDisparity; d++) {
9             if (x - d - radius < 0) continue;
10            int cost = 0;
11
12            // Calculul costului SAD absolut pentru fereastra centrala
13            de dimensiune WxW
14            for (int wy = -radius; wy <= radius; wy++) {
15                for (int wx = -radius; wx <= radius; wx++) {
16                    int leftValue = left.at<uchar>(y + wy, x + wx);
17                    int rightValue = right.at<uchar>(y + wy, x + wx - d
18                );
19
20                cost += std::abs(leftValue - rightValue);
21            }
22
23            // Retinerea disparitatii cu costul cel mai mic (cea mai
24            buna potrivire)
25            if (cost < bestCost) {
26                bestCost = cost;
27                bestDisparity = d;
28            }
29        }
30        disparity.at<uchar>(y, x) = static_cast<uchar>(bestDisparity);
```

```
27 }
28 }
```

Generarea Cadrului Anaglif (Sinteză RGB la Nivel de Sub-Pixel)

```
1 // Initializarea unui cadru cu 3 canale (Blue, Green, Red)
2 Mat anaglyph(leftImage.rows, leftImage.cols, CV_8UC3);
3
4 for (int y = 0; y < leftImage.rows; y++) {
5     for (int x = 0; x < leftImage.cols; x++) {
6         // Preluarea intensitatii luminoase din cadrele sursa
7         uchar leftPixel = leftImage.at<uchar>(y, x);
8         uchar rightPixel = rightImage.at<uchar>(y, x);
9
10        Vec3b color;
11        // Maparea canalelor de culoare pe specificatiile filtrului
12        // Rosu-Cyan
13        color[0] = rightPixel; // Canalul Albastru (Blue) mapat pentru
14        // ochiul drept
15        color[1] = rightPixel; // Canalul Verde (Green) mapat pentru
16        // ochiul drept
17        color[2] = leftPixel; // Canalul Rosu (Red) mapat pentru
18        // ochiul stang
19
20        anaglyph.at<Vec3b>(y, x) = color;
21    }
22 }
```